INITIATION AU LANGAGE MACHINE.

Le langage machine est la suite de bits qui est interprétée par le processeur de l'ordinateur lors de l'exécution d'un programme. C'est le langage natif du processeur, et le seul qui soit reconnu par celui-ci.

Un processeur gère uniquement 2 états (toujours symbolisés par un "1" et un "0") etne peut comprendre un langage rentré par l'utilisateur, un programme python ou java par exemple.

Entre les langages que l'on connait et que l'on implante et le langage machine, il existe des langages intermédiaires et notamment le langage assembleur qui est le langage de plus bas niveau que l'on peut encore appréhender et que le processeur sait traduire en langage machine.

Il existe plusieurs langages assembleurs. On va travailler avec un simulateur et donc de fait utiliser sa syntaxe.

Une instruction machine est une chaîne binaire composée principalement de 2 parties :

- le champ "code opération" qui indique au processeur le type de traitement à réaliser. Par exemple le code "00100110" donne l'ordre au CPU d'effectuer une multiplication.
- le champ "opérandes" indique la nature des données sur lesquelles l'opération désignée par le "code opération" doit être effectuée.

En langage assembleur, on peut rencontrer des calculs simples (addition, soustraction, multiplication), des affectations (placer telle valeur de la mémoire vive dans tel registre et vice versa) et enfin des instructions de saut qui permettent de coder des instructions conditionnelles par exemple.

Il y a trois possibilités pour accéder à un opérande (valeur sur laquelle va être opérée une opération) :

- Il est directement saisi
- Il est dans un registre : il faut alors indiquer le nom de ce registre
- Il est dans la mémoire vive : il faut indiquer son adresse.

1)Syntaxe du langage.

Syntaxe	Signification	Syntaxe	Signification	Syntaxe	Signification
ADD	Addition	LDR	Affectation	HALT	Fin du programme
SUB	Soustraction	STR	Affectation		
MOV	Affectation	B	Aller à		

Comparaison :

CMP	Comparaison CMP R_0 , #10 : compare 10 et la valeur du registre R_0
BEQ	BEQ 20 : Va à l'adresse 20 en mémoire si R_0 =10
BNE	BNE 20 : Va à l'adresse 20 en mémoire si $R_0 \neq 10$
BGT	Va à l'adresse 20 en mémoire si $R_0 > 10$
BLT	Va à l'adresse 20 en mémoire si $R_0 < 10$

Remarque : Pour les instructions de saut, on donnera un Label et non un nombre.

Exemple : BEG test…. Si la condition est vérifiée, le programme sautera les lignes jusqu'à rencontrer test.

2) Exemple d'instructions

MOV R_0, #17 : Affecte la valeur 17 au registre R_0

ADD R_1, R_0, #18. Ajoute 18 à la valeur du registre R_0 et stocke le résultat dans R_1.

LDR R_0, 20 : place la valeur stockée à l'adresse mémoire 20 dans le registre R_0

STR R_0, 20 : place la valeur stockée dans le registre R_0 à l'adresse mémoire 20

Voilà, vous avez quelques syntaxes simples et suffisantes pour la suite. Notez bien que si vous voulez désigner un nombre, le # est nécessaire, sinon vous ferez référence à une adresse de la mémoire.

3) Exercices.

Exercice 1 :

Que signifie les inscriptions suivantes :

a) LDR R1,78 :

b) STR R3, 125 :

c) ADD R1, R0,#128:

d) ADD R0, R1, R2:

e) SUB R1, R0,#128

f) MOV R1, #23

g) B 45 :

h) CMP R0, #23:

i) CMP R0, #23:

BEQ 78:

j) CMP R0, #23:

BNE 78:

k) CMP R0, #23:

BGT 78:

l) CMP R0, #23:

BLT 78:

m) HALT:

Exercice 2 :

Ecrire en langage assembleur les instructions suivantes :

- Additionner 21 avec la valeur du registre R_2 et stocker le résultat dans R_1 :
-
- Placer la valeur stockée à l'adresse 20 dans le registre R_0, lui soustraire 10 puis stocker le résultat dans le registre R_1

Exercice 3 :

Voici une suite d'instructions :

```
0       MOV R0, #4
1       STR R0,30
2       MOV R0, #8
3       STR R0,31
4       LDR R0,30
5       LDR R1,31
6       SUB R2,R0,R1
7       STR R2,158
8       HALT
```

1. Indiquer la signification de cette suite d'instructions. Quel est le contenu de R0 ,R1,R2 à la fin de ces instructions ?
2. Ecrire un programme en langage python qui peut être traduit en langage assembleur avec le code ci-dessus. Les variables correspondent aux adresses mémoires. Par exemple x équivaut à adresse 30, y pour 31, z pour 158.

Exercice 4 :

Voici un code en langage python . Compléter le langage assembleur correspondant :

```
x = 4
y = 8
if x == 10:
        y = 9
else :
        x=x+1
z=6
```

```
MOV ..., #4
   STR R0,30
   MOV …, #....
   STR R0,75
   LDR..., ….
   CMP R0, #...
   BNE else
   MOV R0, #...
   STR R0,75
   B endif
else:
   LDR R0,30
   ADD R0, ……... , # …….
   STR R0,30
endif:
   MOV R0, #.....
   STR R0,23
   HALT
```

Exercice 5 :

Voici un code en langage python :

```
x = 4
y = 8
if(x == y):
    y = x-4
else:
    y = x+y
```

Recopier et compléter le code ci-contre.

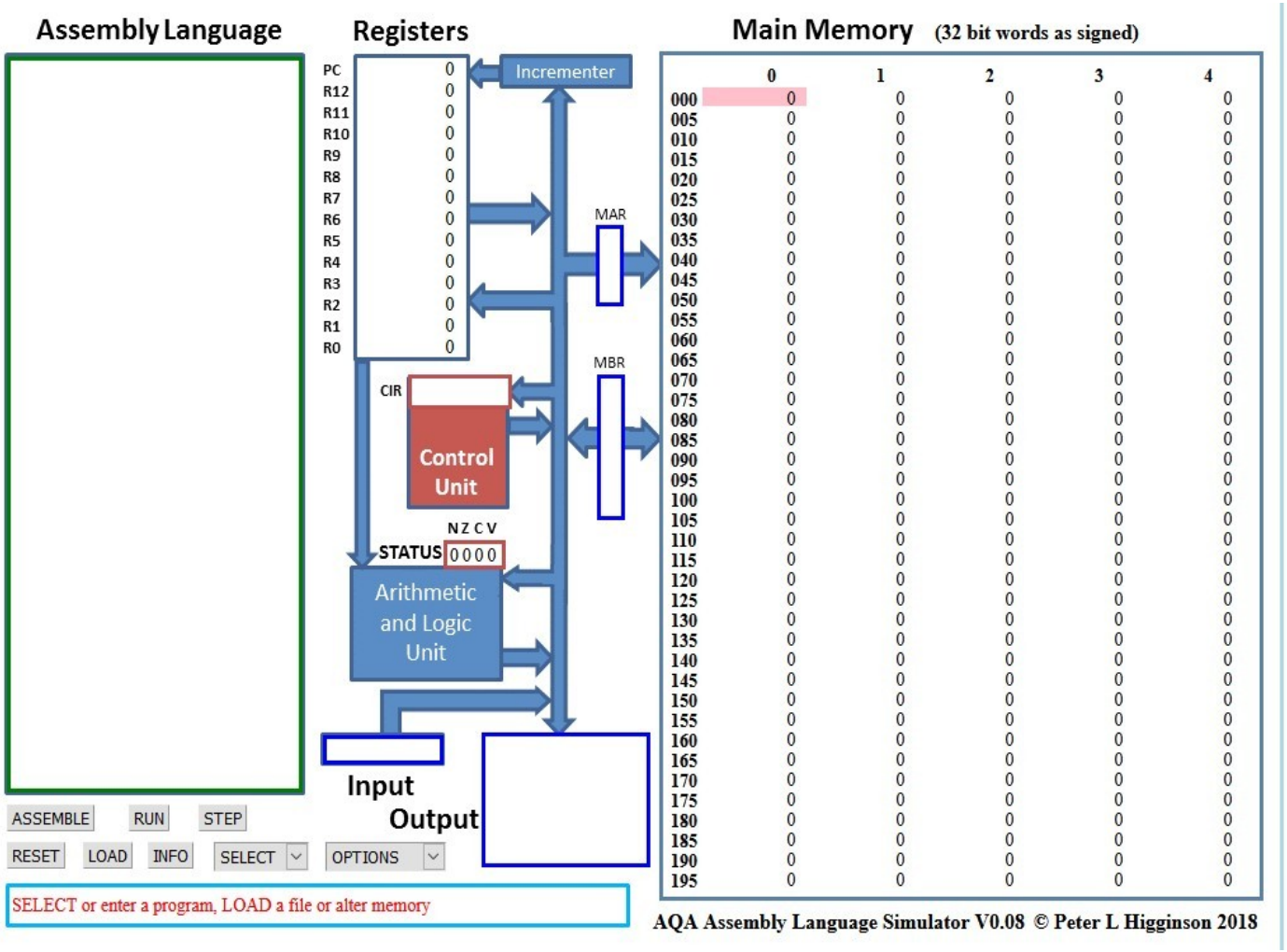
```
      MOV ..., #4
      STR R0,....
      MOV ..., #..
      STR ....
      LDR ...
      LDR ....
      CMP ....
      BNE test
      SUB ....
      .
      .
test:
      ADD ....
      .
      .
      HALT
```

Exercice 6 :

Afin de mettre en pratique ce que nous avons étudié dans le cours "Modèle d'architecture de von Neumann", nous allons utiliser un simulateur développé par Peter L Higginson. Ce simulateur est basé sur une architecture de von Neumann. Nous allons trouver dans ce simulateur :

- une RAM
- un CPU

http://www.peterhigginson.co.uk/AQA/



Il est relativement facile de distinguer les différentes parties du simulateur :

- à droite, on trouve la mémoire vive ("main memory")
- au centre, on trouve le microprocesseur
- à gauche on trouve la zone d'édition ("AssemblyLanguage"), c'est dans cette zone que nous allons saisir nos programmes en assembleur

Revenons sur les parties RAM et CPU

La RAM

Par défaut le contenu des différentes cellules de la mémoire est en base 10 (entier signé), mais d'autres options sont possibles : base 10 (entier non-signé, "unsigned"), base 16 ("hex"), base 2 ("binary"). On accède à ces options à l'aide du bouton "OPTIONS" situé en bas dans la partie gauche du simulateur.

À faire vous-même 1 :

À l'aide du bouton "OPTIONS", passez à un affichage en binaire.
Comme vous pouvez le constater, chaque cellule de la mémoire comporte 32 bits (nous avons vu que classiquement une cellule de RAM comporte 8 bits). Chaque cellule de la mémoire possède une adresse (de 000 à 199), ces adresses sont codées en base 10.
Vous pouvez repasser à un affichage en base 10 (bouton "OPTION"->"signed")

Le CPU

Dans la partie centrale du simulateur, nous allons trouver en allant du haut vers le bas :

- le bloc "registre" ("Registers") : nous avons 13 registres (R0 à R12) + 1 registre (PC) qui contient l'adresse mémoire de l'instruction en court d'exécution
- le bloc "unité de commande" ("Control Unit") qui contient l'instruction machine en cours d'exécution (au format hexadécimal)
- le bloc "unité arithmétique et logique" ("Arithmetic and Logic Unit")

Nous ne nous intéresserons pas aux autres composants de la partie CPU

<u>À faire vous-même 2 :</u>

Dans la partie "éditeur" ("AssemblyLanguage") saisissez les lignes de codes suivantes

```
MOV R0,#42

STR R0,150

HALT
```

Une fois la saisie terminée, cliquez sur le bouton "submit". Vous devriez voir apparaitre des nombres "étranges" dans les cellules mémoires d'adresses 000, 001 et 002 :

Main Memory (32 bit words as signed)

	0	1	2	3
000	-476053462	-443612596	-285212672	0
005	0	0	0	0

L'assembleur a fait son travail, il a converti les 3 lignes de notre programme en instructions machines, la première instruction machine est stockée à l'adresse mémoire 000 (elle correspond à "MOV R0,#42" en assembleur), la deuxième à l'adresse 001 (elle correspond à "STR R0,150" en assembleur) et la troisième à l'adresse 002 (elle correspond à "HALT" en assembleur) Pour avoir une idée des véritables instructions machines, vous devez repasser à un affichage en binaire ((bouton "OPTION"->"binary")). Vous devriez obtenir ceci :

Main Memory (32 bit words as binary)

	0	1	2
000	11100011 10100000 00000000 00101010	11100101 10001111 00000010 01001100	11101111 00000000 00000000 00000000
005	00000000 00000000 00000000 00000000	00000000 00000000 00000000 00000000	00000000 00000000 00000000 00000000

Nous pouvons donc maintenant affirmer que :

- l'instruction machine "11100011 10100000 00000000 00101010" correspond au code assembleur "MOV R0,#42"
- l'instruction machine "11100101 10001111 00000010 01001100" correspond au code assembleur "STR R0,150"

- l'instruction machine "11101111 00000000 00000000 00000000" correspond au code assembleur "HALT"

Au passage, pour l'instruction machine "11100011 10100000 00000000 00101010", vous pouvez remarquer que l'octet le plus à droite, $(00101010)_2$, est bien égale à $(42)_{10}$!

Repassez à un affichage en base 10 afin de faciliter la lecture des données présentes en mémoire.

<u>À faire vous-même 3 :</u>

Pour exécuter notre programme, il suffit maintenant de cliquer sur le bouton "RUN". Vous allez voir le CPU "travailler" en direct grâce à de petites animations. Si cela va trop vite (ou trop doucement), vous pouvez régler la vitesse de simulation à l'aide des boutons "<<" et ">>". Un appui sur le bouton "STOP" met en pause la simulation, si vous rappuyez une deuxième fois sur ce même bouton "STOP", la simulation reprend là où elle s'était arrêtée.

Une fois la simulation terminée, vous pouvez constater que la cellule mémoire d'adresse 150, contient bien le nombre 42 (en base 10). Vous pouvez aussi constater que le registre R0 a bien stocké le nombre 42.



ATTENTION : pour relancer la simulation, il est nécessaire d'appuyer sur le bouton "RESET" afin de remettre les registres R0 à R12 à 0, ainsi que le registre PC (il faut que l'unité de commande pointe de nouveau sur l'instruction située à l'adresse mémoire 000). La mémoire n'est pas modifiée par un appui sur le bouton "RESET", pour remettre la mémoire à 0, il faut cliquer sur le bouton "OPTIONS" et choisir "clr memory". Si vous remettez votre mémoire à 0, il faudra cliquer sur le bouton "ASSEMBLE" avant de pouvoir exécuter de nouveau votre programme.

À faire vous-même 4 :

Modifiez le programme précédent pour qu'à la fin de l'exécution on trouve le nombre 54 à l'adresse mémoire 50. On utilisera le registre R1 à la place du registre R0. Testez vos modifications en exécutant la simulation

À faire vous-même 5 :

Faites en sorte que dans la mémoire 100, il y ait le résultat de la somme de 42 et 54.

À faire vous-même 6 :

Placer les nombres 12 et 15 et 18 aux adresses mémoires respectives 55 et 56 et 57. Pour récupérer une valeur stockée précédemment dans la mémoire, on doit la placer dans un registre. L'instruction est : LDR R0,50(place dans R0 la valeur de la mémoire 50). On souhaite additionner 3 nombres placés dans des mémoires, en n'utilisant que les registres R0 et R1. Placer le résultat à l'adresse mémoire 58.

À faire vous-même 7 :

On souhaite calculer 2^4 en n'utilisant que le registre R0. Il n'y a pas d'instruction "multiplier", il va falloir se débrouiller avec des additions...